EJERCICIO 1 & 2

customers(**customernumber**, customername, contactlastname, contactfirstname, phone, addressline1, addressline2, city, state, postalcode, country, salesrepemployeenumber → employees.employeenumber, creditlimit)

employees(**employeenumber**, lastname, firstname, extension, email, officecode → offices.officecode, reportsto → employees.employeenumber, jobtitle)

offices(**officecode**, city, phone, addressline1, addressline2, state, country, postalcode, territory)

orderdetails(**ordernumber** → orders.ordernumber, **productcode** → products.productcode, quantityordered, priceeach, orderlinenumber)

orders(**ordernumber**, orderdate, requireddate, shippeddate, status, comments, customernumber → customers.customernumber)

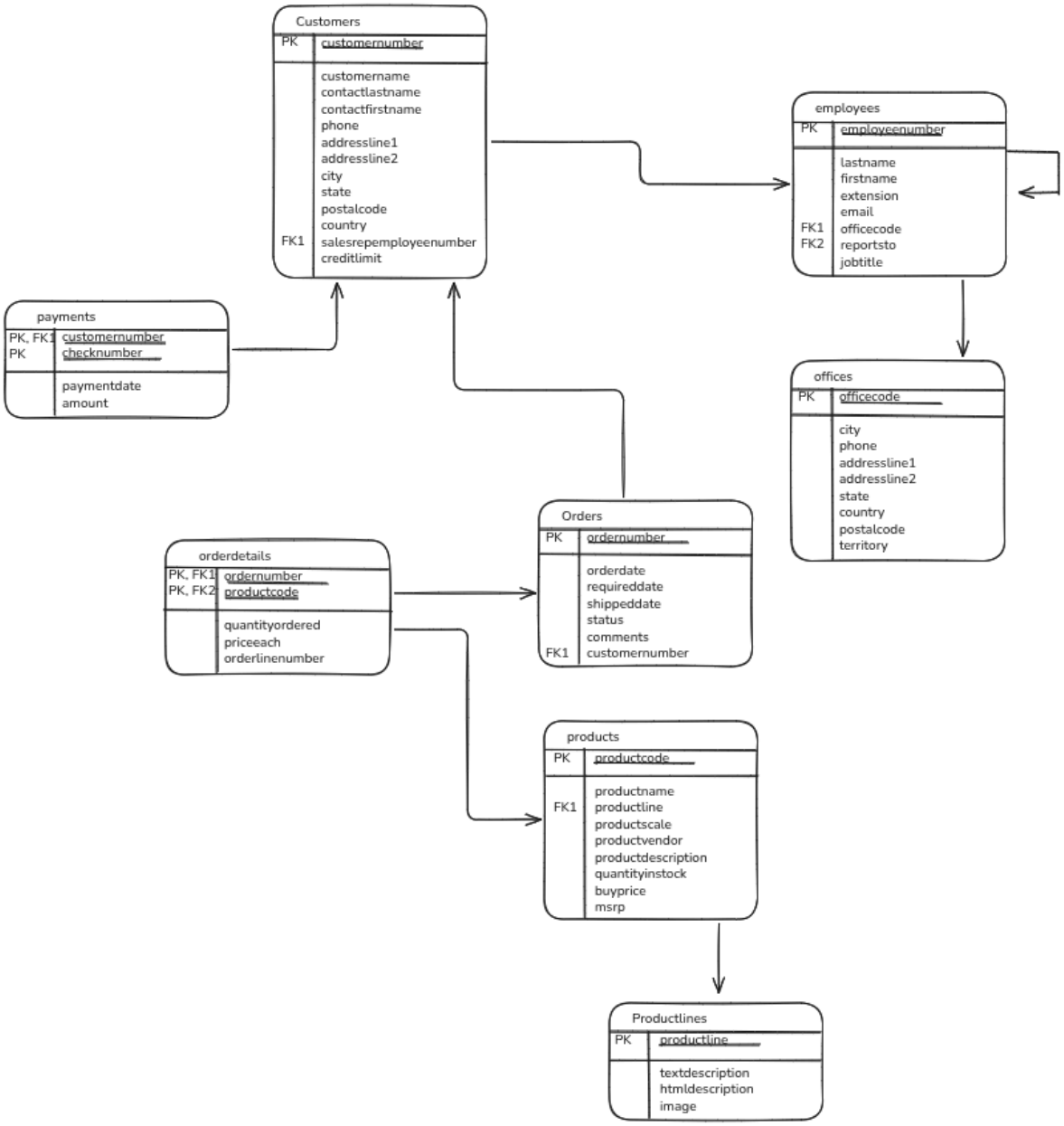
payments(**customernumber** → customers.customernumber, **checknumber**, paymentdate, amount)

productlines(**productline**, textdescription, htmldescription, image)

products(**productcode**, productname, productline → productlines.productline, productscale, productvendor, productdescription, quantityinstock, buyprice, msrp)

Memoria Práctica 1 - Estructuras de Datos
12-10-2026

EJERCICIO 3



EJERCICIO 4

Query 1: Esta consulta requiere una tabla conjunta (usando el comando JOIN) entre ‘customers’ y ‘payments’ para poder mostrar la información de cada cliente y el dinero total abonado, y una condición WHERE aplicada a otra tabla combinada de ‘orders’ y ‘orderdetails’ para filtrar a los clientes (por ‘customernumber’ presente en la tabla ‘orders’) que han comprado el producto dado (1940 Ford Pickup Truck).
Por último, se ordena de mayor a menor por cantidad de dinero abonada (SUM(amount))

query1.sql

```sql
SELECT c.customernumber,
       c.customername,
       Sum(p.amount)
FROM   customers c
       JOIN payments p
         ON c.customernumber = p.customernumber
WHERE  c.customernumber IN (SELECT o.customernumber
                            FROM   orders o
                                   JOIN orderdetails od
                                     ON o.ordernumber = od.ordernumber
                                   JOIN products p
                                     ON od.productcode = p.productcode
                            WHERE  p.productname = '1940 Ford Pickup Truck')
GROUP  BY c.customernumber,
          c.customername
ORDER  BY Sum(p.amount) DESC;

```

Query 2:

En esta consulta nos piden el tiempo medio transcurrido entre que se realiza y se envía un pedido, sorteado por líneas de producto. El tiempo entre pedido y envío se calcula con una resta entre las dos fechas, y para la media usamos la función average, agrupando los resultados por ‘productline’.

```sql
SELECT productline,
       Avg(shippeddate - orderdate)
FROM   orders
       natural JOIN orderdetails
       natural JOIN products
GROUP  BY productline;

```

Query 3:

Para esta consulta hay que hacer un “self-join” al aplicar el comando ‘JOIN’ entre varias tablas ‘employees’ con la FK ‘reportsto’ para crear una tabla conjunta que contenga al jefe de mayor rango (no reporta a nadie) y a sus subordinados directos e inferiores (Hasta 2 niveles inferiores)

query3.sql

```sql
SELECT e3.employeenumber,
       e3.lastname
FROM   employees e1
       JOIN employees e2
         ON e2.reportsto = e1.employeenumber
       JOIN employees e3
         ON e3.reportsto = e2.employeenumber
WHERE  e1.reportsto IS NULL;

```

Query 4: En esta encuesta nos piden la oficina que más productos haya vendido, teniendo en cuenta que un pedido puede ser de más de un producto. Devolvemos el 'officecode', ya que este identifica la oficina, y la suma de productos, asegurándonos de que los pedidos estén en estatus "Enviado" o "Resuelto", los únicos estatus que nos sonaban a que el pago se haya hecho o que el producto se haya entregado, lo cual asumimos que sería la idea detrás de esta encuesta. Los otros estados son cancelados, suspendidos, en disputa y en proceso, ninguno de los cuales suena muy prometedor. No usamos el argumento 'shippeddate' porque no nos da una garantía de que no se haya devuelto el producto luego. Por último ordenamos de forma descendente y limitamos los resultados a uno para que sólo el valor de ventas más alto salga en el resultado.

```sql
SELECT o1.officecode,
       Sum(o2.quantityordered)
FROM   orderdetails o2
       NATURAL JOIN orders o3
       NATURAL JOIN customers c1
       JOIN (employees e1
             NATURAL JOIN offices o1)
         ON c1.salesrepemployeenumber = e1.employeenumber
WHERE  o3.status LIKE 'Shipped'
        OR o3.status LIKE 'Resolved'
GROUP  BY o1.officecode
ORDER  BY Sum(o2.quantityordered) DESC
LIMIT  1;
```

Query 5: En esta encuesta, se deben filtrar los países cuyas oficinas no han vendido nada en un año determinado y su número de oficinas, para ello, se realiza un COUNT(*) de

la tabla de oficinas que cumplen los requisitos por cada país. La forma en la que hacemos esto, es con una tabla anidada en la que se usa WHERE para seleccionar todas las oficinas que no cumplen los requisitos, y un WHERE NOT EXISTS en la tabla principal para excluirlas de los resultados finales.
Un detalle que nos complicó la programación de esta encuesta, es que el valor 'orderdate' de la tabla 'orders' es de tipo DATE en sql, lo que requiere que se haga una operación "WHERE 'date' BETWEEN 'x' AND 'y' " en lugar de "WHERE 'date' = 'x' " que es la que se suele usar para otros tipos de datos y resulta más sencilla

query5.sql

```sql
SELECT o.country, COUNT(*) AS oficinas_sin_ventas
       FROM   offices o
WHERE NOT EXISTS (SELECT DISTINCT e.officecode
                  FROM   employees e
                         JOIN customers c
                           ON c.salesrepemployeenumber =
                              e.employeenumber
                         JOIN orders os
                           ON c.customernumber = os.customernumber
                  WHERE  os.orderdate BETWEEN
                         '2003-01-01' AND '2003-12-31')
GROUP  BY o.country
ORDER  BY oficinas_sin_ventas DESC;

```

Query 6: Esta encuesta está dividida en muchos sub-pasos para hacer el razonamiento más fácil. Se pide que hagamos un listado de todas las parejas que aparecen más de una vez en un carro de la compra, contando en cuantos carros de la compra aparecen y asegurando que no haya duplicados. Productoscomprados encuentra todos los productos que se han pedido, paresdeproductos los empareja, procurando que sean del mismo pedido y que no sean el mismo producto. paresdeproductosfiltrados son los pares de productos que aparecen más de una vez, usando el count cómo condición y devolviendolo tambíen. Por último, paresfiltradísimos asegura que no haya pares duplicados de forma (arg1, arg2), (arg2, arg1), ordenando las salidas usando LEAST y GREATEST y eliminando duplicados con DISTINCT.

```
WITH productoscomprados AS
(
       SELECT *
       FROM   orders natural
       JOIN   orderdetails natural
       JOIN   products), paresdeproductos(codigouno, codigodos) AS
(
       SELECT p1.productcode,
              p2.productcode
       FROM   productoscomprados p1
       JOIN   productoscomprados p2
       ON     p1.ordernumber = p2.ordernumber
       WHERE  p1.productcode != p2.productcode), paresdeproductosfiltrados(codigouno, codigodos, count) AS
(
                SELECT DISTINCT *,
                                Count(*)
                FROM            paresdeproductos
                GROUP BY        codigouno,
                                codigodos
                HAVING          Count(*) > 1), paresfiltradisimos(codigouno, codigodos, count) AS
(
                SELECT DISTINCT Least(codigouno, codigodos),
                                Greatest(codigouno, codigodos),
                                                count
                FROM            paresdeproductosfiltrados)
select *
FROM   paresfiltradisimos;
```

EJERCICIO 5

En este ejercicio nos pedían que arreglaramos 3 fallos con la base de dato classicmodels. Estos eran:

1.Si un empleado se mueve de una oficina a otra se pierde la información de la/s oficina/s donde había trabajado en el pasado

2.Un cliente solo puede relacionarse con un único empleado

3.Los pagos no están asociados a una compra en concreto

Para el primer y segundo problema implementamos una relación entre employees y offices, y customers y employees respectivamente, guardando los primary keys de cada tabla para simbolizar un previo puesto o una comunicación activa. Para saber en qué oficina trabaja un empleado actualmente, guardamos el dateOfEntry (fecha de entrada a esa oficina), la cual podemos sortear de forma descendiente para que el primer elemento sea la oficina actual. Para solucionar el tercer problema añadimos el foreign key ordernumber para que sea parte de la tabla payments, anclando cada pago a un pedido específico

Memoria Práctica 1 - Estructuras de Datos
12-10-2026

